V1.0
Otro texto

Clase 25/03/2026

Buscar para la próxima clase
1. Clave normal
2. Candidata
3. Superclave
4. Clave principal
5. Dependencia funcional

---

functional dependencies:

Definition

It's a generalization of the idea of a key for a relation

Más formalmente: Una dependencia funcional en una relacion R es una proposición de la forma

"Si dos tuplas R son iguales atributo a atributo (tienen los mismos valores en cada componente para cada atributo $A_1, A_2, ..., A_n$, entonces deben de *agree* en otra lista de atributo $B_1, B_2, ..., B_m$"

Debemos ver que con los atributos podemos obtener uno y solo uno de cada una de todas las instancia si seguimos con los mismos atributos para obtener tal.

$$A_1 A_2 \to B_1$$

Quiere decir que todas las instancias de $A_1 A_2$, entonces B_1 debería de tener un único avlor posible

Superkey

A set of attributes that contains a key is called a **superkey**, short for *"superset of a key"*.

Clave candidata

Es una superclave minimal.

Clave Primaria

Es una clave candidata elegida por el diseñador para identificar la celda

Key

An set of attributes it's a key $K \Leftrightarrow K$ functionally determines all other attributes in the table. If you K I can identify the entire tuple

Closure Functional Dependency

It's given some root in a graph. Compute all the dependencys. Then repeat until there's no node added. It's the union of all the nodes in the graph.

Trivial dependencies

$$A_1 \to A_1$$

Transitive rule

$$A_1 \to B_1 \wedge B_1 \to C_1 \Rightarrow A_1 \to C_1$$

Augmentation

$$A_1, ...A_n \rightarrow B_1, ..., B_m \Rightarrow A_1, ...A_n C_1 ..., C_k \rightarrow B_1, ..., B_m C_1 ..., C_k$$

Projecting a Set of Functional Dependencies.

Given

1. A relation R
2. A second relation R_1 (by proyection $R_1 = \pi_{\{L\}}(R)$)
3. Set of functional dependencies that hold in R
4. Compute the closre

Algorithm

For every possible subset of attributes X that exists in your new table S:

1. **Compute the Closure** (X^+)
2. **Intersect with the New Table**: Find the intersection of that closure with the attributes in your new table S. $Y = X^+ \cap S$
3. **Form the New Dependency**: For every attribute A in Y, if A is not already in X, then $X \rightarrow A$ is a functional dependency that holds in the new table.
4. **Simplify**

Avoid redundancy: Normal forms

We want to avoid anomalies in the DDBB because:

- Redundancy: Information repeated. It could cost or be extremaly giant a database if it's not designed correctly
- Update anomalies: we could change the content in one tuple and no in another and it could lead to inconsisteny
- Deletion anomalies: Set of vales becomes empty we could lose Information

Descomposition

Descomposition of a relation R into S and T without redundancy

1. $\{A_i, A_2, ..., A_n\} = \{B_1, B_2, ..., B_m\} \cup \{C_1, C_2, ..., C_k\}$
2. $S = \pi_{\{B_1, B_2, ..., B_m\}}(R)$
3. $T = \pi_{\{C_1, C_2, ..., C_k\}}(R)$

Boyce-Codd Normal form

A relation R is in BCNF $\Leftrightarrow$ whenever there is a nontrivial FD $A_1 A_2 ... A_n \rightarrow B_1 B_2 ... B_m$ for R, it is the case that $\{A_1, A_2, ..., A_n\}$ is a superkey for R.

"Equivalent the left side of every nontrivial FD must contain a key"

Descomposition into BCNF

1. Check whether R is in BCNF. If so, nothing more needs to be done. Return $\{R\}$ as the answer
2. If there are BCNF violations, let one be $X \rightarrow Y$. Use Algorithm 3.7 to compute X^+. Choose $R_1 = R^+$ as one relation schema and let R_2 have attributes X and those attributes of R that are not in X^+.
3. Use Algorithm 3.12 to compute the sets of FD's for R_1 and R_2, let these be S_1 and S_2, respectively
4. Descompose R_1 and R_2 again and again until they are BCNF

- Algorithm 3.7: Closure test

- Algorithm 3.12: Projecting a Set of Functional

Consequences of Descomposition

1. Elimination of anomalies
2. Recoverability of Information
3. Preservation

Formas normales

Una Forma Normal (FN) es un conjunto de reglas o "estándares de calidad" que aplicamos al diseño de una base de datos para asegurar que esté organizada de la manera más eficiente posible.

1st Normal Form

- **Unique Identifies**: we can identify each tuple by some set of components.
- **Atomic values**: components represent unique things. Not many
- Each **column name** must be **unique**
- There must be **no repeating groups**

2nd Normal Form

All data should depend on the whole primary key

3rd Normal Form

The primary key must define all Non Key columns and Non-Key columns must not depend on any other non-key

Formalmente

Whenever $A_1A_2...A_n \rightarrow B_1B_2...B_m$ is a nontrivial FD, either $\{A_1A_2...A_n\}$ is a superkey, or those of $B_1B_2...B_m$ that are not among the $A's$, are each a member of some key (not necessarily the same key).

BCNF

Arriba

Diferencia entre BCNF y 3NF

Es que permite dependencias funcionales hacia claves candidatas (primitivas)

Ejemplo: médico, paciente, especialidad

MVD (Multivalued dependency)

It's is a statement about some rela⊠tion R that when you fix the values for one set of attributes, then the values in certain other attributes are independent of the values of all the other attributes in the relation

$A_1, A_2, ...A_n \twoheadrightarrow B_1, B_2, ...B_m$

For each pair of tuples t and u of relation R that agree on all the $A's$, we can find in R some tuple v that agrees:

1. With both t and u on the $A's$,
2. With t on the $B's$, and
3. With u on all attributes of R that axe not among the $A's$ or $B's$.

Clase 01/04/2026

Estudiar:
- API Rest
- Modelo Relacional
- Algebra Relacional
- Operaciones
- Propiedades de las operaciones

Data model
it's a notation for describing data or Information Consist in
- structure of the data
- Operations on the data: queries and modifications on the database
- Constrains on the data: make the data has some meaning, knowing something that it's represents

important data models
- relation model:
- Semi-structured data model: XML

Relation model
Based on tables (relations). It's as it was an array of C struct, but they are not stored as them Operations: table oriented. know something about all of things in a table Constraints: doens't allow in some components some values

semi Semi-structured
it's a tree of graphs. Represent data hierachically nested tagged elements.

operations: follow paths in a tree

Constraints:
- data type of values associated with a tag.
- which data could be nested to a given tag.

Other data models
Object orintation models

. values can have structure
- Relations can have associated methods

Comparisons between data models
we want it to be fast. in that case, we preffered relational model

Relational Model
Represent data using relations: two-dimensions table

Attributes
Names of columns in a relation. They appear at the top of a table. Describes meaning of the column below.

Schemas

Name of a relation and the set of attributes. Attributes in a schema it's a set. But to talk about relations we need some standard, so we treat it like lists.

why? I don't see the benefits

Tuples

Rows of a relation. Tuples has one ***component*** for each attribute of the relation

Domains

each component should be atomic → must be of some type. Cannot be a record structure like array, set, etc.

The order in which we mention the attributes it's irrelevant.

Relation instances

a database changes over time. It has added, updated or deleted tuples. And, rarely the schema could change. An instence it's in some time the version of the database.

keys of relations

Set of attributes forms a key ⇔ we do not allow two tuple in the relation in all the attributes of the key.

Defining relation schema in SQL

SQl consists in: Data Definition and Data manipulation (for queries)

There are three kinds of relations:

- Stored relations (tables). The ones we use commonly.
- Views: relations defined by a computation. They are constructed when needed.
- Temporary tables: constructed by SQL language processor when it performs its job executing queries and data modifications

```
CREATE TABLE name (
  attribute_name   type
);
```

Data types

- STRING
  - CHAR: fixed length string
  - VARCHAR: fixed length string up to n characters.

Difference it's just implementation dependent

- Bit strings
  - BIT: bit strings of length n
  - BIT VARYING (n): bit strings of length up to n
- Boolean:
  - BOOLEAN: can take
    - True
    - False
    - Unknown
- INT / INTEGER: number

- SHORTINT

Difference its implementation dependent

- DATE
- TIME
- FLOAT / REAL
- DOUBLE (more precission)
- DECIMAL: decimal number that takes up to n decimal digits

Modifying relation schema

```
DROP TABLE table_name
```

ALTER TABLE

Has two options:

- ADD
- DROP

```
ALTER TABLE table_name option attribute
```

Default Values

When creating `DEFAULT value`

Declaring keys

- PRIMARY KEY
- UNIQUE: if the key consists in more than one attribute

Algebraic Query language

It's useful because it's less powerfull than common languages

What's an algebra?

operator and atomic operands

relation algebra: atomic operands are

- variables
- constants

Operators

- Set operations:
  - union
  - intersection
  - Difference
- Selection of some tuples
- Proyection of some columns
- Operations to combine two relations
  - Cartesian Product
  - Joins
- Renaming of attributes. Doesn't affect tuples, but to a schema

Set operations
union $R \cup S$ Intersection $R \cap S$ Difference $R - S$

1. R and S must have schemas with identical set of attributes. And types (domains) for each attribute must have the same in R and s
2. Colums R and S must be ordered before, so the order is same for both relations

Proyection
Produce from a relation R a new relation that has only some of R's columns

Selection
Produces a new relation with a subset of R's tuples. the tuples are those who satisfy some condition C.

Cartesian Product
Makes the cartesian product. Everyone meets everyone (talking about tuples)

Natural Joins
Given two relations we don't want all combinations, only pair tuples that match in some way $R \bowtie S$. We join them by the attributes that are common to each one. All rows generated are called: *joined tuples*

If there are multiple rows that have the same values in both, it gives all combinations taking all the rows

Theta Joins
it's less restrictive than natural joins, giving us actually a way to take the tuples that we want.

1. Take the cartesian product
2. Select from the product only those tuples that satisfy the condition C

Combining Operations to Form Queries
Put the result of some operation to another operation.

Renaming attributes
To make it easier in some queries it's useful to change the name of the attributes

Relationships Among Operations

$$R \cap S = R - (R - S)$$

$$R \bowtie_C S = \sigma_C(R \times S)$$

The natural join of R and S can be expressed by starting with the product $R \times S$. We then apply the selection operator with a condition C of the form

$$R.A_1 = S.A_1 \quad \text{AND} \quad R.A_2 = S.A_2 \quad \text{AND} \quad \ldots \quad \text{AND} \quad R.A_n = S.A_n$$

$R \bowtie S = \pi_L(\sigma_C(R \times S))$

Constraints on Relations
restrict the data that may be stored in a database

Relational Algebra as a Constraint Language
- $R = \emptyset$: There are no tuple in the result of R

- $R \subseteq$ Every tuple in R must be also in S

Referential Integrity Constraints
A value appearing in one context also appears in another, related context

$\pi_A(R) \subseteq \pi_B(S)$ (some attribute A and B)

Key Constraints
If an attribute (like name) is a Key, then every name must be unique. This implies that if two rows have the same name, they must be the exact same row (i.e., they must have the same address).

Using the notation above, it's possible to define precisely the constraints for keys.

1. Self-Product
2. Filter for Violations: Use Selection (σ)
3. the result of this search must be the Empty Set ($\emptyset$).

$$\sigma_{\text{MS1.name} = \text{MS2.name} \wedge \text{MS1.address} \neq \text{MS2.address}}(\text{MS1} \times \text{MS2}) = \emptyset$$

Clase 15/04/2026

Aprender sobre:
1. ACID
2. Transacciones

Además, voy a leer el capítulo 4 para ir adelantando

Las bases de datos están en casi todos los sitios web.

El poder viene de DBMS (database managment system)

They are useful for: creating and managing large amounts of data efficiently and allowing it to persist over long periods of time,

The Evolution of Database System s

DBMS expected to:
1. Allow users to create new databases and specify their schemas, using data-definition language
2. Give users the ability to query the data (question about data) and modify data, using data-manipulation language.
3. Support the storage of very large amounts of data over a large period of time
4. Enable durability, recovery of database in face of failure, erros of many kind or intentional misuse

+Control access to data from many users at once, without allowing unex pected interactions among users (isolation) and without actions on the data to be performed partially but not completely (atomicity)

Early Database Management Systems

First commertial DBMS
1. Late 1960´s. Evolved from file systems. Do not guarantee that data cannot be lost if it's not backed up
2. Do not support data-manipulation language.
3. Used in:
   1. Banking systems
   2. Airline reservation systems
   3.

Relational Database Systems

Paper of Codd proposed:

Database systems should present the user with a view of data organized as tables called relations

- Programmers should not be concerned for the way the data is stored

Smaller and Smaller Systems

Apparition of XML
Large collections of small documents can serve as a database, and the methods of querying and manipulating them are different

Bigger and Bigger Systems
Need to store more and more information. Pentabytes of information. Example: google, satelites information, Peer-to-peer file-sharing systems

Information integration
Many distributed information to tell the same thing:

Solution:
1. creation of data warehouses, where information from many legacy databases is copied periodically, with the appropriate translation, to a central database.
2. Middleware: sup⊠port an integrated model of the data of the various databases, while translating between this model and the actual models used by each databas

Overview of a Database Management System
two distinct sources of commands to the DBMS:

1. Conventional users and application programs that ask for data or modify data.
2. A database administrator: a person or persons responsible for the structure or schema of the database.

DDL (Data Definition Language)
schema-altering data-definition language (DDL) commands are parsed by a DDL processor and passed to the execution engine, which then goes through the index/file/record manager to alter the metadata, that is, the schema information for the database

Answering the Query
query parsed and compiled by: ***query compiler***

To perform to answer the query is passed to the ***execution engine*** This ***requests*** small pieces of data ***to: resource manager***. This knows about: data files, format and size of records and index files.

Request passed to: ***buffer manager***. Bring appropriate portions of data where it is kept permanently

Transaction Processing
Queries and DML actions grouped into ***transacitons***:

Units that must be ***executed atomically*** and in ***isolation*** Durable:
1. concurrency-control manager or scheduler: ensures atomicity and isolation.
2. logging and recovery manager, responsible for the durability of transactions.

Storage and Buffer Management
Data in: ***secondary storage***. Usually, it control storage on the disk directly.
1. The storage manager keeps track of the location of files on the disk and obtains the block or blocks containing a file on request from
2. The buffer manager is responsible for partitioning the available main memory into buffers.

Information that may be needed.
1. Data: contents of database itself
2. metadata: database schema. Describes schema
3. Log records: information about recent changes.
4. Statistics: information gathered and stored by DBMS
5. Indexes: data structures to access efficiently to the memory.

Transaction Processing

transaction manager therefore accepts transaction commands from an application. Tells when transactions begin and end

Transaction processor steps:

1. Logging: assure durability: writes the log in buffers and negotiates with the buffer manager to make sure that buffers are written to disk
2. Concurrency control: the scheduler (concurrency-control manager) must assure that the individual actions of multiple transactions are executed in such an order that the net effect is the same as if the transactions had in fact executed in their entirety, one-at-a-time
3. Deadlock resolution:The transaction manager has the responsibility to intervene and cancel ("roll back" or "abort") one or more transactions to let the others proceed.

ACID:

1. Atomicity: all-or-nothing execution of transaction
2. Consistency: consistency constraints, or expectations about relationships among data elements
3. isolation: each transaction must appear to be executed as if no other transaction is executing at the same time
4. Durability: the effect on the database of a transaction must never be lost, once the transaction has completed

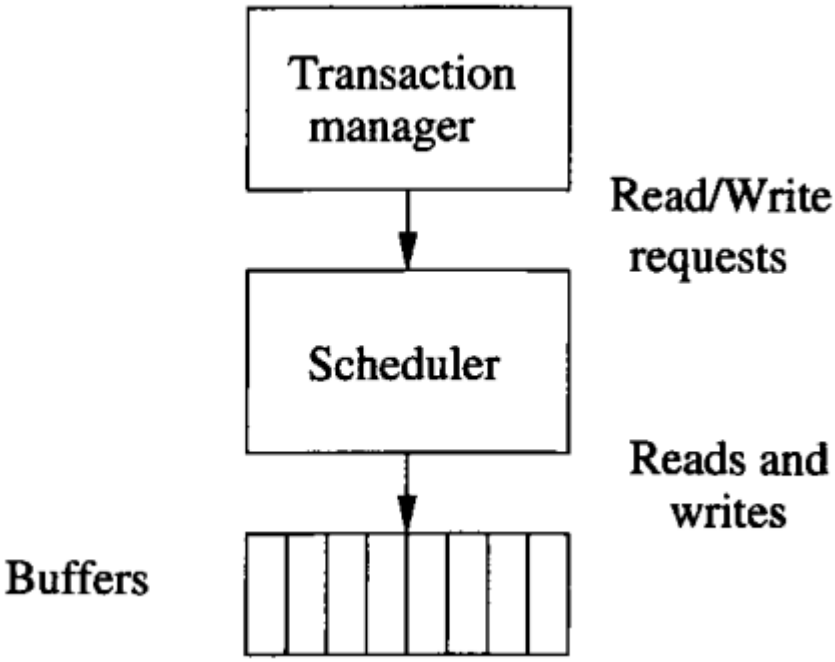
The Query Processor

1. The query compiler: translates the query into an internal form called a query plan.
    1. Query parser: tree structure from the textual form of the query.
    2. Query preprocessor: semantic checks on the query and performing some tree transformations
    3. Query optimizer: transforms the initial query plan into the best available sequence of operations on the actual data.
2. Execution engine: executing each of the steps in the chosen query plan

End of Chapter 1

Concurrency Control (Chapter 18)

Must preserve consitency. Timing of individual steps must be regulated: ***scheduler***. Process of assuring transacitons preserve consistency when executed simultaneously: ***concurrency-control***



In most cases, the scheduler will execute write and reads directly. But, some situations: delay request or even abort the request.

To assure transactions preserve correctness of the database state: ***serializability*** and stronger: ***conflict-serializability*** .

locking, timestamping, and validation

Serial and Serializable Schedules

"correctness principle": every transaction, if executed in isolation (without any other transactions running concurrently), will transform any consistent state to another consistent state

Schedules

It's is a sequence of the important actions taken by one or more transactions. Place: main memory, not disk.

Database element read by multiple queris

T_1	T_2
READ(A,t)	READ(A,s)
t := t+100	s := s*2
WRITE(A,t)	WRITE(A,s)
READ(B,t)	READ(B,s)
t := t+100	s := s*2
WRITE(B,t)	WRITE(B,s)

Clase 22/04/2026

Buscar para la próxima clase
1. Transaction manager
2. Scheduler
3. Data manager
   1. Recovery manager
   2. Cache manager

Cap. 17, 18

Issues to address:
1. Data must be protected in the face of a system failure
2. Data must not be corrupted simply because several error-free queries or database modifications are being done at once.

Technique for resilience:
1. log
   1. Records securely history of database changes
   2. Undo/redo actions

Things that could go wrong

Failure modes

Erroneous data entry
Write constraints and triggers that detect data beliveed to be erroneous

Media fialures
Local failure of disk (only some bits). Detected by parity checks

Head crashes, all disk unreadable
1. RAID shcemes. Lost disk may be restored
2. Mantian an archive. Copy of database, distributed among several sites

Catastrophic failure
Media holding database completely destroyed
1. explossions

Solutions: archiving and redundant

System failure
System failures are problems that cause the state of a transaction to be lost

typical erros:
1. Power loss
2. Software erros

Solution. logs all database changes in separte nonvolatile log and redundancy of that info.

more about transactions
The transaction is the unit of execution of database operations.

Begins with: as soon opreatins on database are executed and end with “COMMIT” or “ROLLBACK”

Must:
1. atomically: all or nothing

Transaction manager job. transactions are executed correctly.
1. issuing signas to log manager. necessay information in the form of “log records” can be stored on the log.

Assuring that concurrently executing transactiosn do not interfere with each other in ways that introduce errors

he log manager maintains the log. It must deal with the buffer manager, since space for the log initially appears in main-memory buffers, and at certain times these buffers must be copied to disk

Correct Execution of Transactions
Database compose of “elements”: value that can be accessed or modified by transactions

Usually are one or more of:
1. Relations
2. Disk blocks or pages
3. Individuals tuples or objects

Database State: value for each of its elements.

The **Correctness Principle**: If a transaction executes in the absence of any other transactions or system errors, and it starts with the database in a consistent state, then the database is also in a consistent state when the transaction ends.

1. Transaction atomic
2. Execute simultaneously precaution

Operations of transactions
There address spaces that interact in important way
1. Space of disk blocks holding database elements
2. Virutal or main memory managed by buffer manager
3. Local address space of transaction

Read database element:
1. Brought to main memory buffer/s. Content of buffer can be read by transaction into its own address space

Writing:
1. Created the new value in its own space.
2. Value copied to appropiate buffer

Buffer may or not be copied to disk immediately. That’s responsability of buffer manager.

Primitives:
1. INPUT(X): copy disk block containing database element X to memory buffer
2. READ(x,t) copy database element X to transaction local variable t.
3. WRITE(X, t): copy value of local variable t to database element X in memory buffer
4. OUTPUT(X): copy block containingX from its buffer to disk

constraints
1. Databse element no larger than single block.
2. READ and WRITE issued by transactions

3. OUTPUT and INPUT issued by buffer manager

Undo logging

log is a file of log records, each telling something about what some transaction has done

Undoing the effects of transactions that may not have completed before the crash

Log Records

Log as file opened for appending only.

log manager job of recording in the log each important event.

Logs:

1. <START >: transaction T has begun.
2. <COMMIT>: Transaction T has completed successfully and will make no

more changes to database elements. All changes should appear on disk, but we cannot be sure when we sure it that it's already there.

1. <ABORT>: Transaciton T could not complete successfully
2. <T, X, v>: transaction T has changed database element X and its former value was v.

Undo-Logging Rules

1. If transaction T modifies database element X , then the log record of the form < T ,X ,v> must be written to disk before the new value of X is written to disk.
2. If a transaction commits, then its COMMIT log record must be written to

disk only after all database elements changed by the transaction have been written to disk, but as soon thereafter as possible

And summarizing the rules before: transactions materal must be written to disk in following order:

- The log records indicating changed database elements. (each element individdually)
- The changed database elements themselves. (each element individdually)
- The COMMIT log record.

The essential pol icy for undo logging is that we don't write the <COMMIT T> record until the OUTPUT actions for T are completed.

Two cases:

1. Transaction with commit record seen. Do nothing. Must not be undone
2. T incomplete transaction or aborted transaction. Recovery manager must change the value of X in database to V.

Recovering Crashes are idempotent.

Checkpointing

Problem: we do not keep ALL history logs, only the last ones. But since when? Checkpointing

- Stop accepting new transactions.
- Wait until all currently active transactions commit or abort and have

written a COMMIT or ABORT record on the log.

- Flush the log to disk.
- Write a log record , and flush the log again.
- Resume accepting transactions.

Nonquiescent Checkpointing
allows new transactions to enter the system during the checkpoint

Steps:

1. Write log record <Start CKPT (T1, ...) Tk> and flush log
2. Wait unitl all of T1, ... Tk commit or abort.

When all of T1, ..., Tk have completed, write <END CKPT> and flush log

Redo Logging

Redo vs undo logging
1. While undo logging cancels the effect of incomplete transactions and ig⍰nores committed ones during recovery, redo logging ignores incomplete transactions and repeats the changes made by committed transactions
2. While undo logging requires us to write changed database elements to disk before the COMMIT log record reaches disk, redo logging requires that the COMMIT record appear on disk before any changed values reach disk
3. old values of changed database elements are exactly what we need to recover when the undo rules U
   and U2 are followed, to recover using redo logging, we need the new values instead.

Write ahead logging rule

Order:
1. The log records indicating changed database elements.
2. The COMMIT log record.
3. The changed database elements themselves

Order of Redo matters (from eariliest to latest)

Recover using redo

1. Identify committed transactions
2. Scan log forward from beginning
   1. For each log record
      1. If T is not commited transaction: nothing
      2. If T is commited transaction, write value v for database element X.

Undo / redo loggin

Rules

update log record that we write when a database element changes value has four components. Record < T ,X ,v,w > means that transaction T changed the value of database element X; its former value was v, and its new value is w

- before modifying database element. Update record < T, X, v, W >
- < COMMIT T> log record can precede or follow any of the changes to the database elements on

disk.

Recovery

1. Redo all the committed transactions in the order earliest-first, and
2. Undo all the incomplete transactions in the order latest-first.

Checkpointing

1. Write a < START CKPT (T1, .. . , Tk)> record to the log, where T1,… , Tk are all the active transactions, and flush the log.
2. Write to disk all the buffers that are dirty; i.e., they contain one or more

changed database elements. Unlike redo logging, we flush all dirty buffers, not just those written by committed transactions.

1. Write an < END CKPT > record to the log, and flush the log

Protecting Against M edia Failures

The archive

Archiving: copy of database separate from database itself.

TO mantain it the most same as the original, we could use logs and send them when created

1. Full dump: entire database copied
2. Incremental dump: only those database elmeent changed, after *full dump*

   Nonquiescent Archiving

   nonquiescent dump copies the database elements in some fixed order,

possibly while those elements are being changed by executing transactions.

1. Write a log record < START DUMP>
2. Perform a checkpoint appropriate for whichever logging method is being used.
3. Perform a full or incremental dump of the data disk(s), as desired, making sure that the copy of the data has reached the secure, remote site.
4. Make sure that enough of the log has been copied to the secure, remote site that at least the prefix of the log up to and including the checkpoint in item (2) will survive a media failure of the database.
5. Write a log record < END DUMP>.

Recovery Using an Archive and Log

1. Restore the database from the archive.
   1. Find the most recent full dump and reconstruct the database from it (i.e., copy the archive into the database).
   2. If there are later incremental dumps, modify the database according to each, earliest first.
2. Modify the database using the surviving log. Use the method of recovery appropriate to the log method being used.

Concurrency control

Transaction manager and scheduler Scheduler: regulate of timing of indiviaul steps of different transactions.

serializability is stronger: Conflict-serializability (used)

Serial and serializable schedules

Schedules

schedule is a sequence of the important actions taken by one or more trans⊠actions

Serial Schedules

Schedule serial if its actions consist of all the actions of one transaction, then all the actions of another transaction, and so on. No mixing of the actions is allowed.

schedule S is serializable if there is a serial schedule S' such that for every initial database state, the effects of S and S' are the same

The Effect of Transaction Semantics

Any database element A that a transaction T writes is given a value that depends on the database state in such a way that no arithmetic coincidences occur

Notation for Transactions and Schedules

- Actions are:
  - R_T(X): reads database element X
  - W_T(X): writes database element X
- Transaction T_i sequence of actions with subscript i
- Schedule S of set of transaction T is a sequence of actions in which for each transaction Ti in T , the actions of Ti appear in S in the same order that they appear in the definition of Tj itself.

Conflic serializability

It's based on the idea of a conflict: a pair of consecutive actions in a schedule such that, if their order is interchanged, then the behavior of at least one of the transactions involved can change.

Conflicts

Any two actions of different transactions may be swapped unless:

1. They involve the same database element, and
2. At least one is a write

Conflict-equivalent if they can be turned one into the other by a sequence of nonconflicting swaps of adjacent actions.

Precedence Graphs and a Test for Conflict-Serializability

T1 takes precedence over T2 if there are actions such that:

1. Ai is ahead of A2 in S,
2. Both A
   and A2 involve the same database element, and
3. At least one of A
   and A2 is a write action

Use this precedence as a graph. Nodes: transactions of schedule S

Not necessary

Case: Wi(Y); wi(X); w2(Y); w2(X); w^X); And this schedule: wi(Y); w2 (Y); w2 (X); Wi(X); w3 (X);

We construct precedence graph and ask is there are any cycles.

1. Acyclic: S is conflict serializable
2. Cyclic: not

Enforcing Serializability by Locks

Scheudler: uses locks to prevent unserializable behavior.

Locks

Transactions must request and release locks

Notation:
1. u_l(X): unlock
2. l_u(X): lock

Use of locks proper
- Consistency of Transactions:
  1. ransaction can only read or write an element if it previously was granted a lock on that element and hasn't yet released the lock.
  2. transaction locks an element, it must later unlock that element

1. Legality of schedules: Locks must have their intended meaning: no two transactions may have locked the same element without one having first released the lock

The Locking Scheduler
Scheduler job: grant requests if and only if the request will result in a legal schedule

Scheduler only modifies this relation Locks (element, transaction),

Two-Phase Locking (2PL)
Guarantee that a legal schedule of consistent transactions is conflict⊠serializable

1. In every transaction, all lock actions precede all unlock actions.

Locks are obtained and seconds pahse: locks are relinquised.

Locking System s W ith Several Lock M odes

Shared and Exclusive Locks
1. sl_i(X) transaction T_i request a shared lock on database element X
2. xl_i(X) for T_i request an exclusive lock on X

Requiriments
1. Consistency of transactions: ransaction may not write without holding an exclusive lock, and you may not read without holding some lock.
   1. A read action ri(X) must be preceded by sli(X) or xk(X), with no intervening Ui(X).
   2. A write action Wi(X) must be preceded by xli(X), with no interven⊠ing Ui{X).
2. Two phase locking of transactions: Locking must precede unlocking
3. Legality of schedules: element may either be locked exclusively by one transaction or by several in shared mode, but not both
   1. xl_i(X): appears in schedule: cannot be following xl_j or sl_j for some j other than i wihtout an u_i(X)
   2. sl_i(X) appears in schedule then there cannot be following xl_j(X) for $j \neq i$ without intervening u_i(X)

Compatibility Matrices
A convenient way to describe lock-management policies. It has a row and column for each lock mode. The rows correspond to a lock that is already held on an element X by another transaction, and the columns correspond to the mode of a lock on X that is requested.

Upgrading Locks
Problems with multiple shared locks, causing a deadlock.

Update Locks

Fix problem with Update lock. update lock uli(X) gives transaction T j only the privi⍰lege to read X , not to write X

Only update lock can be updated to write lock later. Read lock doesn't.

Can grant update lock on X when there are update locks. Prevents additionals locks of any kind when there's an update lock on X.

Increment locks

Operate on database only by incrementing or decrementing stored values. Tehy commute with each other. (a + b) = b + a

Architecture Scheduler

Principles

1. Transactions do not request locks or cannot be relied upon to do so
2. Transactions do not release locks

Mantains a lock table.

Clase 29/04/2026

Buscar para la próxima clase:
1. Plan de ejecucion:
   1. Qué es,
   2. Como se decide cual es,
   3. Como se ejecuta.

Cap 15, 16

---

query processor is the group of components of a DBMS that turns user queries and data-modification commands into a sequence of database operations and executes those operations

Preview of Query Compilation

Three steps:
1. parsing: creation of parse tree for query
2. Query optimization:
   1. Query rewrite: parse tree converted to initial query plan. Then transformed into another equivalente plan (logical query plan)
   2. Physical plan generation.

For the query optimization we need to decide: algebraically equivalent forms, what algorithm should we use to implement that operation and how should operations pass data from one to the other.

Introduction to Physical-Query-Plan Operators

Scanning tables

Most basic thing in physical query plan:
1. Basic: Read entire contetns of relation R.
2. Variation: involves simple predicate. We read only those tuples of the relation R that satisfies predicate.

Approaches:
1. *Table scan*: blocks containing tuples of R are known to the system and it's possible to get the blocks one by one.
2. *Index scan*: Use index to get all tuples of R. We can use it to get all the tuples that have particular values.

Sorting while scanning tables

May include `BY`. SOme approaches required one or both arguments to be sorted relations

Sort scan: takes relation R and specification of attributes on which sort is to be made and produces R in that sorted order.

The Computation M odel for Physical Operators

1. Arguemnts of any operator on disk, result of operator is left in main memory.
2. Result of a part of query often is not written to disk.

Parameters for Measuring Costs

Statistics / Parameters express cost of operator.

1. Number of buffers used for a query
2. Number of tuples in R. $T(R)$
3. Number of blocks needed to hold all tuples of R. $B(R)$
4. Number of distinct values that appear in a column of a relation. $V(R, a)$

I/O Cost for Scan operators

Number of I/O needed:

1. Table-scan
   1. Relation R clustered: B
   2. Relaiton not clustered, R distributed among tuples of other relations. Table Scan: T
2. Index scan:
   1. Many fewer than B(R). More I/O than examining entire index.

Iterators for Implementation of Physical Operators

Operations:

1. Open Initializes data structures needed to perform the operations.
2. GetNext. Returns next tuple in result and adjusts data structures as necessary.
3. Close: ends iterations after all tuples have benn obtained.

One-Pass algorithms

Execute each of individual steps

1. Sorting based
2. Hash based
3. Index based

ALogrithms for operators in degress of difficulty and cost

1. One pass: read data only once from disk.
2. Two pass: reading data a first time from disk, process, write all to disk and read a second time for further processing. Too large to fit in main memory, but not largest.
3. Three or more passed. Wihtout limit of size of data. Generalization of two pass

Operators:

1. Tuple at a time, unary operations (selection and projection). Not entire R. Read block at a time, use main memory buffer and prodoce output.
2. Full relation, unary operations. Seiing all or most of tuples in memory at once. Size M (main memory buffers available)
3. Full relations. BInary operations. At least one arguments to be limited to size M to use one-pass algorithm

One pass algorithms for tuple at a time operations

1. Read blocks
2. Perfomr operation on each tuple
3. Move selected tuples to output buffer

Disk I/O requirements as in scan-io

One-Pass Algorithms for Unary, Full-Relation Operation

1. Duplicate elimination
   1. Read each block of R one at a time and decide wheter we saw it or not the tuple
   2. Need: keep in memory one copy of every tuple we have seen.
2. Grouping:
   1. Min MAX : store the value so far found

    2. COUNT: add one for each tuple of the group seen
    3. SUM(a). add the value
    4. AVG. Mantain 2 accumulaitons: count of number of tuples and sum of a-values of these tuples.

Then produce output for each group
1. We cannot produce output until the last tuple is seen.

One-Pass Algorithms for Binary Operations
1. Set union ($R \cup S$)
    1. S into M-1 buffers
    2. Build search structure.
    3. Copy all S into output
    4. For each tuple t of R
        1. If it's not in S, write to output
        2. Otherwise no
2. Set intersection
    1. S into M-1 buffers
    2. Build search structure (key = tuple)
    3. Read each block of R and for each tuple t
        1. If t is in S, copy t to output
        2. If not, ignore
3. Set difference
    1. R is larger relation
    2. Read S into M - 1 buffers
    3. Build search structure with full tuples as key
4. Bag intersection
    1. S into M-1 buffers. Each distinct tuple a count. Copies of tuple t not stored individually.
    2. Read each block of R
    3. For each tuple t of R see whetet t occurs in S
        1. If not, ignore t
        2. If yes, output t and decrement count by 1.
            1. If t appears in S, but with count 0, ignore it
5. Bag difference
    1. Read tuples of S into main memory and count number of occurrences
    2. For each tuple t in R
        1. See if it occurs in s
            1. If true, decrement associated count
    3. Copy in main memory all tuples whose count is positive
6. Product:
    1. S into M-1 buffers
    2. For each tuple t of R
        1. Concatenate t with each tuple of S in main memory
7. Natural Join
    1. Read all tuples of S and form them into search structure with attributes Y (intersection between them) as search key
    2. Read each block of R.
    3. For each tuple t of R
        1. Find tuples of S that agree with t on all attributes of Y
        2. Form a tuple by joining it wiht y and move the resulitng tuple to output.

Nested-Loop Joins

One and a halp passes. A variation will be read once and the other will be read many

Tuple based nested loop join

```
FOR each tuple s in S DO
  FOR each tuple r in R DO
    IF r and s join to make a tuple t then
      output t;
```

Iterator for tuple based nested loop join

Allows us to avoid storing intermediate relations on disk in some sitautions.

BLOCK Based nested loop join algorithm

1. Organizing access to both argument relations by blocks, and
2. Using as much main memory as we can to store tuples belonging to the relation 5, the relation of the outer loop.

TODO

Two-Pass Algorithms Based on Sorting

1. It's usually enough.
2. Generlization it's easy

Two-Phase, Multiway Merge-Sort

1. Phase 1. Fill M buffers iwth tuples form R and sort them. Write sorted sublist to secondary storeage
2. Phase 2: Merge sorted sublists

Merge sorted sublists into one sorted list:

1. Smallest key among first remianing elements of all lists.
2. Smallest element to first available position of output block
3. If output blokc full write to disk and reinitialize same buffer in main memory
4. If block was just taken is exahusted of records, read next block. If no blocks remian, leave its buffer empty.

Duplicate Elimination Using Sorting

In the way it sorts, when t is wrote in the output, it drops the top if it's the same as t

Grouping and Aggregation Using Sorting

1. Read the tuples of R into memory
2. One main buffer for each sublist and load the first block of each sublist into its buffer
3. Loop: Find least value of sort key present among first avaialbe tuples.
   1. Prepare to compute all aggregates on list L.
   2. Examine each of tuples with sort key v
   3. Buffer becomes empty, replace it with next block form same sublist

Sort-based union algorithm (R and S)

1. Created sorted sublists from R and S
2. Use 1 main memory buffer for each sublist of R and S.
3. Repeatedly find the first remaining tuple t among all the buffers. Copy t to the output, and remove from the buffers all copies of t

Sort-Based Intersection and Difference

Repeatedly consider the tuple t at least in the sorted order among all tuples remaining. Produce output as follows and then remove lal copies of t from the input buffers.

A Simple Sort-Based Join Algorithm

1. Sort R and S using 2PMMS
2. Merge sorted R and S

Two-Pass Algorithm s Based on Hashing

Hash all tuples of the argument using hash key. . We reduce the size of operands using a factor equal to number of buckets.

Partiioning relations by hashing

h hash function. Associate one buffer with each bucket. Bucket copied to appropriate buffer.

Hash based algorithm for duplicate

```
initialize M-l buckets using M-l empty buffers;
FOR each block b of relation R DO BEGIN
  read block b into the Mth buffer;
  FOR each tuple t in b DO BEGIN
    IF the buffer for bucket h(t) has no room for t THEN
      BEGIN
        copy the buffer to disk;
        initialize a new empty block in that buffer;
      END;
    copy t to the buffer for bucket h(t);
  END;
END;
FOR each bucket DO
  IF the buffer for this bucket is not empty THEN
    Write the buffer to disk;
```

Hash-Based Grouping and Aggregation

Saving Some Disk I/O's

Hash S, Choose to keep m of the k buckets in main memory.

Index-Based Algorithms

Clustering and NonClustering indexes

Clustering indexes: indexes on an attribute or attributes such that all tuples with fixed value for search key of this index appear on roughly as can hold them.

Index based selection

Index-Based Algorithms

TODO

Joins Using a Sorted Index

TODO

Buffer Management

Task: central task of making main-memory buffers avaialbe to process

Allow proceses to get the memory they need, minimizaing delay and unsatisfiable requests.

Buffer Management Architecture

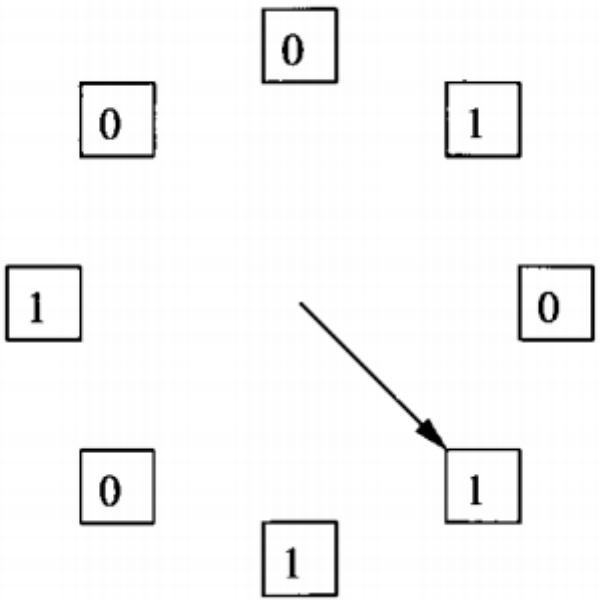
1. The buffer manager control memory directly,
2. allocate in virtual memory, allowing the operating system to decide which buffers are actually in main memory at any time and which are in the "swap space" on disk that the operating system manages

We assume that there is a fied-size buffer pool

Buffer Management Strategies

What block to throw out.

1. LRU (least recently used)
2. FIFO
3. Clock algorithm



4. System Control: query processor give advice to buffer manager to avoid some mistaked that would occur with strict policy.

The Relationship Between Physical Operator Selection and Buffer Management

Can the algorithm adapt to changes in valud of M

1. When expected M buffers are not avialbable. How does the buffer replacement strategy used impact the number of I/O's performed

Algorithms Using More Than Two Passes

Generalization of 2 pass Algorithms

Multipass sort based algorithm

Simple recursive approach

1. M main memory to sort relation R (clustered)

Basis: R fits in M blocks, read R into amin memory, sort and write sorted relation to disk.

Induction: R does not fin into main memory, partiition blocks holding R. Sort all R_i, merge M sorted sublists.

Multipass Hash-Based Algorithms

Basis: unary operation: relation fits in M buffers, read it into memory and perform operion. BInary operaiton: either relation in M-1 buffers, perform Induction: No relation fits in main memory. Hash each relation into M-1 buckets, perform the operation on each bucket or corresponding pair of buckets and accumulate output

Performance of Multipass Hash-Based Algorithms

TODO

Query compiler

1. Query, parsed
2. Parse tree into expression tree of relational algebra. It's logical query plan
3. Logical qiery turned into physiccal query plan.

Parsing and preprocessing

Sintax analysis and parse trees

ATOMS: SELECT, atrtibutes or relations, constants, parentheses, operators and other schema elements.

1. Syntactic categories: names for families of query subparts. < query >, < condition >

Node is atom. Has no children. However, node is syntacitc cateogry, then its children are described by one of the rule.

A Grammar for a Simple Subset of SQL

Queries

< Query > ::= SELECT < SelList > FROM < FromList > WHERE < Condition > < SelList > ::= < Attribute > , < SelList > < SelList > ::= < Attribute>

< FromList > ::= < Attribute > , < FromList > < FromList > ::= < FromList >

< Condition > ::= < Condition > AND < Condition > < Condition > ::= < Attribute > IN (< Query >)
< Condition > ::= < Attribute> = < Attribute> < Condition > ::= < Attribute> LIKE < Pattern >

Preprocessor

Several important functions

1. Check relation uses: every relation in a FROM cluase
2. Check and resolve attribute uses: every attribute that is mentioned in SELECT, WHERE clause must be an attribute of some relation in current scope.
3. Check types: all attributes must be of type appropriate to their uses.

Preprocessing queries involving views

Preprocessor needs to replace operang by piece of parse tree that represents how the view is constructed from base tables.

we can push selections and projections down the tree, and combine them in many cases

Algebraic Laws for Improving Query Plans

Commutative and Associative Laws

1. $R \times S = S \times R; (R \times S) \times T = R \times (S \times T)$
2. $R \bowtie S = S \bowtie R; (R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$
3. $R \cup S = S \cup R; (R \cup S) \cup T = R \cup (S \cup T)$

4. $R \cap S = S \cap R; (R \cap S) \cap T = R \cap (S \cap T)$
5. $R \bowtie_C S = S \bowtie_C R$

It's nos associative: $R \bowtie_{R.b>S.b} S) \bowtie_{a<d} T$

For bags, the laws for sets can differ

Laws involving selection

1. $\sigma_{C_1} \text{AND}_{C_2}(R) = \sigma_{C_1}\left(\sigma_{C_2}(R)\right)$
2. $\sigma_{C_1} \text{OR}_{C_2}(R) = \left(\sigma_{C_1}\right) \cup \left(\sigma_{C_2}(R)\right)$ only if it's a set
3. For a union, the selection must be pushed to both arguments
4. For a difference, the selection must be pushed to the first argument and optionally may be pushed to the second
5. For the other operators it is only required that the selection be pushed to one argument. For joins and products, it may not make sense to push the selection to both arguments, since an argument may or may not have the attributes that the selection requires.
6. $\sigma_C(R \times S) = \sigma_{C(R)} \times S$
7. $\sigma_C(R \bowtie S) = \sigma_{C(R)} \bowtie S$
8. $\sigma_C(R \bowtie_D S) = \sigma_{C(R)} \bowtie_D S$
9. $\sigma_C(R \cap S) = \sigma_{C(R)} \cap S$

Pushing Selections

Queries involve virtual views: necessary to move selection as far up the tree as it can go and then push selections down all possible branches

Trivial Laws

1. Any selection on an empty relation is empty
2. C is an always true condition, then $\sigma_{C(R)} = R$
3. R is empty, then $R \cup S = S$

Laws involving projections

Principle: We may introduce a projection anywhere in an expression tree, as long as it eliminates only attributes that are neither used by an operator above nor are in the result of the entire expression.

1. $\pi_L(R \bowtie S) = \pi_{L(\pi_M(R) \bowtie \pi_N(S))}$
2. $\pi_L(R \bowtie_C S) = \pi_{L(\pi_M(R) \bowtie_C \pi_N(S))}$
3. $\pi_L(R \times S) = \pi_{L(\pi_M(R) \times \pi_N(S))}$

Laws About Joins and Products

1. $R \bowtie_C S = \sigma_C(R \times S)$
2. $R \bowtie S = \pi_L(\sigma_C(R \times S))$: C condition that equates each pair of attributes from R and S with same name and L is list that includes one attribute from each equated pair and all other attributes of R and S

From right to left. Much faster

Laws Involving Duplicate Elimination

1. $\delta(R) = R$ if R has no duplicates

2. $\delta(R \times S) = \delta(R) \times \delta(S)$
3. $\delta(R \bowtie S) = \delta(R) \bowtie \delta(S)$
4. $\delta(R \bowtie_C S) = \delta(R) \bowtie_C \delta(S)$
5. $\delta(\sigma_C(R)) = \sigma_C(\delta(R))$

Laws Involving Grouping and Aggregation

1. $\delta\big(\gamma_{L(R)} = \gamma_L(\pi_M(R))\big)$ if M is a list containing at least all those attributes of R mentioned
2. $\gamma_{L(R)} = \gamma_{L(\delta(R))}$

From Parse Trees to Logical Query Plans

Query with condiiton with no subqueries. Replace entire construct by relaitonal algebra expression

Removing Subqueries From Conditions

< Condition > that has a subquery. Intermediate form of operator.

Operator: two argument selection

Node: left child that represents the relation R and right child expression

Suppose: Two argument selection

1. Replace < condition > by the tree that is the expression for S.
2. Replace the two argument selection by a one-argument selection σ_C where C is condition that equates each component of the tuple t to the corresponding attribute of the relation S
3. Give σ_C argument that is the product of R and S

Improving the Logical Query Plan

1. Selections can be pushed down the expression tree as far as they can go. If a selection condition is the AND of several conditions, then we can split the condition and push each piece down the tree separately
2. Projections down the tree.
3. Duplicate eliminations be removed or moved to more convenient position in tree
4. Selections be combined with product below to turn pair of operations into equijoin.

Grouping Associative/Comm utative Operators

1. Replace natural joins with theta joins
2. Add projections to eliminate duplicate copies
3. Theta join conditions must be associative.

Estimating the Cost of Operations

Need to see:

1. Order and grouping for associative and Commutativ operations
2. Algorithm for each operator
3. Additional operators needed
4. Arguments passed from one operator to next

Estimating Size of intermediate relations

We want rules so:

1. Give accurate estimates
2. Easy to compute
3. Logically consistent

estimating size of projections

It may increase or decrease the size.

Estimating size of selections

Reduce number of tuples, size of tuples remain the same.

Estimating size of join

TODO

Natural Joins W ith Multiple Join Attributes

TODO

Joins o f M any R elations

TODO

Estimating size for other operations

TODO

Introduction to Cost-Based plan selection

TODO

Obtaining Estimates for Size Parameters

TODO

Computation of Statistics

TODO

Heuristics for Reducing the Cost of Logical Query Plans

TODO

Approaches to Enumerating Physical Plans

TODO

Choosing an Order for Joins

TODO

Significance of Left and Right Join Arguments

TODO

Join trees

TODO

Left-Deep Join Trees

TODO

Dynamic Programming to Select a Join Order and Grouping

TODO

Dynamic Programming W ith More Detailed Cost Functions

TODO

A Greedy Algorithm for Selecting a Join Order

TODO

Completing the Physical-Query-Plan

TODO

Choosing a Selection Method

TODO

Choosing a Join Method

TODO

Pipelining Versus Materialization

TODO

Pipelining Unary operations

TODO

Pipelining Binary Operations

TODO

Notation for Physical Query Plans

TODO

Ordering of Physical Operations

TODO